CSC10014 · PRACTICAL TRACK · 2026

Lab 6

Adding AI to the Assistant, Safely

What an LLM is, a mini search engine, open-book answers and saying "I don't know"

Practical Track · Week 6 · 3–3.5 h · No prior AI experience needed

Faculty of Information Technology · University of Science, VNU-HCM

Instructor: Dr. LE Thanh Tung

Contents

About This Lab

Lab 6 — Adding AI to the Assistant, Safely. Practical track, Week 6. Duration: 3–3.5 hours in class, plus a short homework.

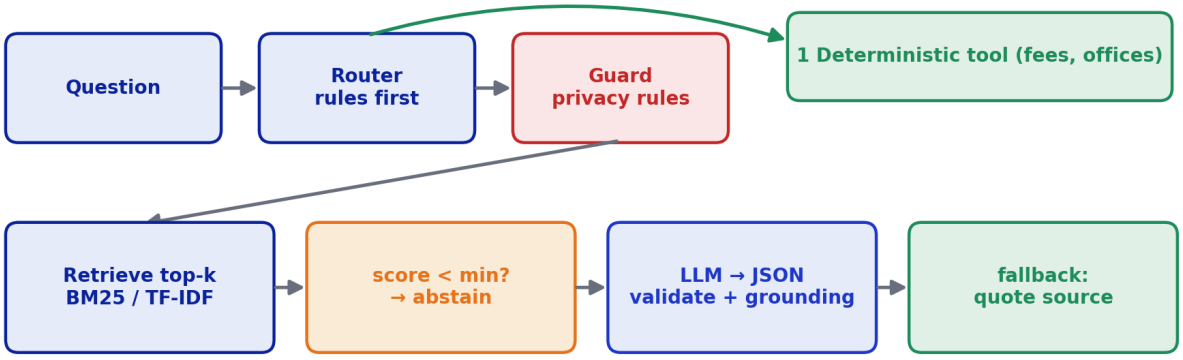
This lab assumes you have **never studied AI, machine learning or datasets**. You know programming, DSA and OOP — and that is enough. We explain what a language model is in plain words, build a small search engine with dictionaries and sets, and then let a *fake* model answer questions, so everything works on your laptop without internet or payment. Using a real model is an optional last step.

What you already know → what is new today

You already know	Today you learn	The link
dict, set, sorting (DSA)	a **search index**	an inverted index is a dict: word → set of documents
scoring and sorting items	**ranking** search results	score each document, sort by score
a function's return type	**structured output**	we force the AI to return data in a fixed shape
input validation (Labs 3–4)	**checking AI answers**	never trust AI output without checking it
a class behind an interface (Lab 3)	an **LLM adapter**	a fake model and a real model share one interface

Goal

Add a third way of answering to the assistant. Until now it could (1) compute fees and look up offices with **rules**. Today it also (2) **finds** the right paragraph in the official documents, and (3) lets a **language model** phrase an answer from that paragraph — and it learns to say **"I don't know"** instead of inventing things.



2 retrieval path · 3 LLM path — each with a defined failure behaviour

Figure 0.1 — The three paths. Every path has a safe behaviour when something goes wrong.

What you hand in

Level	Task	Where
Must	warm-up S1–S3: a mini search engine (`practice/mini_search.py`) — tests green	branch
Must	assistant tasks A1–A2 (`starter/`) — tests green	PR
Must	the comparison table from `eval/compare_paths.py` with 3 sentences	PR
Should	homework H1: add 5 test questions and re-run the comparison	PR
Bonus	run a real model with Ollama, or improve one failed question	PR

Timeline

Time	Activity
0:00–0:25	Part A: what is an LLM? (no code)
0:25–1:00	Part B: build a mini search engine (S1–S3)
1:00–1:25	Part C: from mini search to the real knowledge base
1:25–1:35	break
1:35–2:15	Part D: the fake model, structured output, the grounding check (A2)
2:15–2:45	Part E: saying "I don't know" (A1)
2:45–3:10	Part F: compare the three paths
3:10–3:20	submit; explain the homework

Before Class

1. Lab 5 works: tests pass and feedback survives a restart.
2. Unzip the Lab 6 materials; you will use `lab06/practice/` and `lab06/starter/`.
3. Set up the starter once:

```
$ cd lab06/starter
$ python -m venv .venv
$ source .venv/bin/activate            # Windows: .venv\Scripts\activate
(.venv) $ pip install -r requirements.txt
(.venv) $ pip install -e .
(.venv) $ cp .env.example .env
```

4. Optional, only for the bonus: install **Ollama** (ollama.com) and run `ollama pull llama3.2` at home (about 2 GB download).

IN OUR COURSE

Everything required today works **offline** with a fake model called the *stub*. You do not need an API key or to pay for anything. Never paste API keys into code or chat.

Part A — What Is an LLM? (No Code)

A.1 In one sentence

A **large language model (LLM)** — the technology inside ChatGPT, Gemini or Claude — is a program trained on a huge amount of text to **predict the next word**. Repeating that prediction word after word produces fluent sentences.

Everyday picture	What it means for us
a very well-read student who answers instantly	great at explaining and rephrasing
who never admits "I don't know" unless told to	can invent a rule or a number that sounds right
who has not read *our* faculty's documents	does not know *our* current rules
who may answer slightly differently each time	results are not perfectly repeatable

> **WARNING**
> An LLM can be **confidently wrong**. It may invent a deadline or a fee. In a student assistant that is worse than saying nothing. Most of today is about preventing this.

A.2 Words you will hear

Word	Plain meaning
model	the trained AI program
prompt	the text we send to the model: instructions + question
token	a piece of a word; models read and write tokens, and cost is counted in tokens
hallucination	a fluent answer that is not true
retrieval	searching our own documents for the relevant paragraph
RAG	*retrieval-augmented generation*: search first, then let the model answer from what was found

A.3 RAG = an open-book exam

Asking an LLM directly is a **closed-book** exam: it answers from memory. RAG turns it into an **open-book** exam: we first find the right page in *our* documents, give it to the model, and tell it: *"answer only from this page — or say you don't know."*

Step	Who does it	Kind of work
find the right paragraph	**our search code**	DSA: dict, set, sort
write a friendly answer from it	the language model	AI
check the answer cites that paragraph	**our code**	validation

Step	Who does it	Kind of work
no good paragraph? say "I don't know"	**our code**	a simple `if`

Notice: three of the four steps are ordinary programming you already know.

CHECK YOURSELF

1) Why can't we just ask a normal chatbot about our faculty's thesis rules?

2) In RAG, which step does the model do?

3) What is a hallucination?

Part B — Build a Mini Search Engine (S1–S3)

Before any AI, we need **search**: given a question, find the most relevant paragraph. Open `lab06/practice/mini_search.py`. It has four tiny "documents":

```
DOCS = {
    "D1": "Students may register a thesis with at least 120 credits and a GPA of 2.0.",
    "D2": "Retake credits are charged at 1.5 times the normal credit rate.",
    "D3": "A lost student card can be replaced at Student Affairs in room I.102.",
    "D4": "Students may borrow up to 5 books from the library for 14 days.",
}
```

B.1 Step 0 — words (given)

`tokenize` turns text into a list of lower-case words and removes very common words (*stop words* such as "the", "a", "my") that do not help search:

```
>>> tokenize("I lost my student card!")
['lost', 'student', 'card']
```

B.2 S1 — the inverted index

To find documents quickly, we build an **inverted index**: a dict from each word to the **set** of documents that contain it — exactly like the index at the back of a book.

```
{"thesis": {"D1"}, "credits": {"D1", "D2"}, "card": {"D3"}, "library": {"D4"}, ...}
```

```
def build_index(docs: dict[str, str]) -> dict[str, set[str]]:
    index: dict[str, set[str]] = {}
    for doc_id, text in docs.items():
        for word in tokenize(text):
            index.setdefault(word, set()).add(doc_id)
    return index
```

B.3 S2–S3 — score and rank

A simple score: **how many words of the question appear in the document**. Then sort, best first.

```
def search(index, query, k=2):
    scores: dict[str, int] = {}
    for word in tokenize(query):
        for doc_id in index.get(word, set()):
            scores[doc_id] = scores.get(doc_id, 0) + 1
    return sorted(scores.items(), key=lambda kv: (-kv[1], kv[0]))[:k]
```

```
>>> idx = build_index(DOCS)
>>> search(idx, "I lost my student card")
```

```
[('D3', 3)]
>>> search(idx, "how many credits for a thesis")
[('D1', 2), ('D2', 1)]
>>> search(idx, "what is the capital of France")
[]
```

The last result is important: **no document matches**, so an honest assistant should say "I don't know". Remember this for Part E.

```
(.venv) $ cd lab06/practice
(.venv) $ pytest -q test_mini_search.py
....                                                                     [100%]
4 passed
```

EXERCISE

Try `search(idx, "how many books can I borrow")`. Which document wins, and with what score? Now try `"borrowing books"` — why does it fail? (Hint: "borrowing" ≠ "borrow".)

NOTE

Counting shared words is a good start, but real search engines also give more weight to **rare** words ("thesis" tells more than "students") and less to very long documents. The starter uses **BM25**, a famous formula that does exactly this — you use it, you do not need to derive it.

Part C — The Real Knowledge Base

Go to `lab06/starter/`. The documents are in `data/kb/` as three Markdown files: a student handbook, a fee document and campus services. They are **sample** documents written for this course.

C.1 A document, and its metadata

```
---
doc_id: H26
title: Student Handbook 2026 (sample)
version: 2026.1
owner: Training Office
---
# 5.3 Thesis registration
Students may register a thesis when they have at least 120 accumulated credits and a
cumulative GPA of at least 2.0. Registration opens in week 1 and closes in week 3 of the
semester.
```

The part between `---` lines is **metadata**: which document, which version, who owns it. It lets the assistant say *where* an answer comes from.

C.2 Chunks

We cut each document into small pieces called **chunks** — one per section — and give each an id such as `H26#5.3`. The assistant searches chunks, not whole documents, so it can point to the exact paragraph.

```
(.venv) $ python
>>> from pathlib import Path
>>> from assistant_core.rag.chunker import load_folder
>>> chunks = load_folder(Path("data/kb"))
>>> len(chunks), chunks[0].id, chunks[0].title
(14, 'F26#1', '1 Tuition per credit')
```

C.3 Search with BM25 (given)

```
>>> from assistant_core.rag.index import make_index
>>> idx = make_index("bm25", chunks)
>>> [(h.chunk.id, h.score) for h in idx.search("can I register my thesis with 118 credits",
2)]
[('H26#5.3', 4.914), ('H26#5.4', 2.449)]
>>> idx.search("what is the capital of France")
[]
```

The score is higher when more (and rarer) words match. We will use a **minimum score** (`RETRIEVAL_MIN_SCORE=2.0` in `.env`): below it, the paragraph is probably not relevant.

> **EXERCISE**
> Search for three questions of your own. Write down the top chunk id and score. Which one surprised you?

Part D — The Language Model, Safely

D.1 A fake model to learn with: the stub

Real models need internet, a key or a big download, and answer differently each time. To learn, test and grade, we use **`StubLLM`**: a tiny class that *pretends* to be a model. It reads the first paragraph we give it and answers from its first sentence — always the same way.

It follows the same interface as a real model (you know this from Lab 3):

```
class LLM(Protocol):
    name: str
    def complete(self, prompt: str) -> str: ...
```

So the rest of the code does not care whether it talks to the stub, to Ollama on your laptop, or to an online model. Switching is one line in `.env`:

```
LLM_PROVIDER=stub       # stub (default, offline) | ollama (local model) | openai (online,
needs a key)
```

D.2 The prompt is a text file

Open `src/assistant_core/prompts/answer_v1.txt`. This is what we send to the model:

```
You are the study assistant of the Faculty of Information Technology.
Answer ONLY from the SOURCES below. If the sources do not contain the answer, set "abstain" to
true.
Never invent rules, numbers or dates. Keep the answer under 80 words.
Return ONLY a JSON object, no markdown:
{"answer": "<text>", "source_ids": ["<id>", ...], "confidence": <0..1>, "abstain":
<true|false>}

SOURCES:
{sources}

QUESTION: {question}
```

Our code replaces `{sources}` with the chunks found by search and `{question}` with the student's question. Keeping the prompt in a file means you can improve the wording without touching Python code.

D.3 Structured output: a fixed shape for the answer

We ask the model for **JSON with four fields**, and we describe that shape with a Pydantic class — like a function's return type:

```
class AnswerDraft(BaseModel):
    answer: str
    source_ids: list[str] = []
    confidence: float = Field(ge=0, le=1)      # between 0 and 1
    abstain: bool = False                      # True = "I don't know"
```

If the model returns something else — plain text, broken JSON, a confidence of 7 — validation fails, and we do not show it to the student.

D.4 A2 — the grounding check (2 lines)

One more check is essential: the model must cite **only chunks we actually gave it**. If it cites `H26#9.9`, a section that does not exist, it probably invented the answer.

Open `src/assistant_core/llm.py`, find `parse_draft`, and complete **TODO A2**:

```
    if not draft.abstain and (not draft.source_ids or not set(draft.source_ids) <=
allowed_ids):
        raise ProviderError("answer cites sources that were not retrieved")
```

In words: *if the model does not say "I don't know", it must cite at least one source, and every cited id must be one we retrieved.* `set(a) <= set(b)` means "every element of a is in b" — a set operation from DSA.

Run the grounding test:

```
(.venv) $ pytest -q tests/test_rag.py -k citation
.                                                                    [100%]
```

D.5 What happens when the model fails?

The given `RagAnswerer` (in `src/assistant_core/rag/answerer.py`) tries the model, checks the answer, and tries **once more** if the check fails. If it still fails, it falls back to quoting the best paragraph directly — not as nice, but always true:

Situation	What the student sees	`path`
model answer passes all checks	a friendly answer + source	`llm`
model returns garbage twice	the paragraph quoted + source	`retrieval` (degraded)
model is offline	the paragraph quoted + source	`retrieval` (degraded)

Part E — Teaching the Assistant to Say "I Don't Know" (A1)

E.1 Why before the model?

If search finds nothing relevant, a real model will still write a fluent answer — from its memory, not our rules. That answer might be wrong, and it costs time and money. So we stop **before** calling the model.

E.2 A1 (2 lines)

In `answerer.py`, complete **TODO A1**: if there are no hits, or the best score is below the minimum, return the "I don't know" response.

```
        if not hits or hits[0].score < self.min_score:
            return Response(answer=ESCALATE_MSG, intent=Intent.UNKNOWN, path="abstain",
escalate=True,
                            confidence=0.0)
```

`ESCALATE_MSG` politely tells the student who can help (the Training Office). This is called **escalating** to a human.

E.3 Run everything

```
(.venv) $ pytest -q
.....................................                    [100%]
42 passed
(.venv) $ python -m assistant_core "I lost my student card"
According to 3 Student card replacement: A lost student card can be replaced at Student
Affairs (room I.102).
  source: S26#3 — 3 Student card replacement
(.venv) $ python -m assistant_core "what is the capital of France"
I'm not confident I can answer that from the official documents. Please contact the Training
Office
(room I.101) or ask your academic advisor.
(.venv) $ python -m assistant_core "what grade did my friend get"
I can't share information about other students. You can only see your own records after
logging in.
```

The last answer comes from a **guardrail** (`guard.py`): a few simple rules that stop questions about other students' data *before* search or AI. It is given; read it.

E.4 And the fees?

`"tuition for 18 credits and 2 retake"` still goes to the **fee calculator** from Lab 3 — the router sends it there before search. Money must be computed exactly by code, never by a language model.

NOTE

The starter also contains a small example of **tool calling** (`run_tool_call` in `tools.py`): a model may *ask* for the fee calculator with JSON such as `{"tool": "calc_fee", "args": {"credits": 18}}`, and our code checks the arguments and runs the real function. You do not need to change it today — just know that the model only *asks*; our code *decides and executes*.

Part F — Compare the Three Paths

Which path is best? We do not guess — we **measure**. The file `eval/cases.jsonl` has 20 test questions with the expected source, or `"expect_abstain": true` when the right behaviour is "I don't know":

```
{"id": "E01", "question": "can I register my thesis with 118 credits", "expect_source":
"H26#5.3"}
{"id": "E03", "question": "tuition for 18 credits and 2 retake", "expect_contains":
"9,450,000"}
{"id": "E16", "question": "what is the capital of France", "expect_abstain": true}
```

```
(.venv) $ python eval/compare_paths.py
path                correct  abstain ok  p50 ms  max ms
deterministic          7/20       5/5     0.0     0.5   fails: E01 E02 E05 E06 E07 E08 E09 E10
E11 E12 E13 E14 E15
retrieval             16/20       4/5     0.0     0.1   fails: E13 E14 E15 E19
llm (stub)            16/20       4/5     0.1     0.4   fails: E13 E14 E15 E19
```

F.1 How to read the table

Column	Meaning
correct	questions answered with the right source (or correctly refused)
abstain ok	of the 5 "should say I don't know" questions, how many it refused
p50 ms / max ms	typical and slowest time per question

- **deterministic** (rules only) never invents anything — but answers only fees and offices.
- **retrieval** answers many more by quoting documents.
- **llm** answers as many and phrases them nicely; with a real model it becomes slower (seconds instead of milliseconds).

F.2 Why do E13, E14, E15, E19 fail?

These failures are left on purpose — they are typical, and fixing them is how real teams improve:

Case	Question	What goes wrong
E13	"when is tuition due"	the documents say "paid by the end of week 6", not "due" — different words
E14	"can I take 32 credits"	"credits" ≠ "credit" for our simple word matching
E15	"how long can I keep a library book"	the **router** sees "library" and answers with the library's room
E19	"is the final exam open-book"	a related paragraph (exam absence) scores high; the stub cannot know it should refuse

EXERCISE

In `docs/lab6/path_comparison.md`, copy the table and write three sentences: which path you would use for handbook questions, why, and one failure you would fix first.

Bonus — A Real Model on Your Laptop (Ollama)

If you installed Ollama and downloaded a model at home:

```
$ ollama serve                          # in its own terminal (often starts automatically)
$ LLM_PROVIDER=ollama LLM_MODEL=llama3.2 python -m assistant_core "can I register my thesis
with 118 credits"
$ LLM_PROVIDER=ollama LLM_MODEL=llama3.2 python eval/compare_paths.py
```

Compare with the stub: is it slower? Does E19 now say "I don't know"? Did anything get worse? Write what you observe — real models vary.

> **TIP**
> The first question is slow because the model loads into memory. Later questions are faster.

Team Integration

1. Each member opens a PR with A1–A2 and their comparison table.
2. The team merges one version and keeps the **deterministic** and **retrieval** paths — you will compare against them in Week 7.
3. Member 1 tags `v0.6-ai`.

Homework (about 1 hour)

H1 — Five more test questions (should)

Add **five** questions to `eval/cases.jsonl` about your team's domain or the sample documents:

- three normal questions with the expected source (look it up in `data/kb/`),
- two questions the assistant **should refuse** (`"expect_abstain": true`) — for example something not in the documents, or about another person.

Re-run `python eval/compare_paths.py` and paste the new table. Did any of your questions fail? Write one sentence about why.

```
{"id": "E21", "question": "how many books can I borrow", "expect_source": "S26#1", "category":
"normal"}
{"id": "E22", "question": "who will win the football match", "expect_abstain": true,
"category": "out-of-scope"}
```

H2 — Reflection (must, 5 lines)

Explain RAG to a friend who has never studied AI, using the open-book exam picture. Mention one thing our code checks that the model does not.

H3 — Bonus: fix one failure

Pick E14 ("can I take 32 credits"). In `src/assistant_core/rag/index.py`, make `tokenize` remove a final "s" from words longer than four letters. Re-run the comparison: does E14 pass now? Did anything else change?

How to Submit

What	When	How
S1–S3 (practice) + A1–A2, tests green	end of the session (draft PR OK)	PR link on the LMS
comparison table + three sentences	end of the session	`docs/lab6/path_comparison.md` in the PR
team merge, tag `v0.6-ai`	23:59 the day before Lab 7	Tech Lead: tag link
H1 five questions + new table; H2 reflection	23:59 the day before Lab 7	PR link on the LMS

Grading checklist

Criterion	Points
S1–S3 mini search tests pass	2
A2 grounding check — tests pass	2
A1 abstain before the model — tests pass	2
comparison table + three sentences	1
H1 five questions added, table re-run	2
H2 reflection	1
Bonus (Ollama run or H3 fix)	+1
Total	**10 (+1)**

Troubleshooting

Problem	What to do
`NotImplementedError: A1 ...`	TODO A1 in `answerer.py` is not finished
a test about citations fails	TODO A2 in `llm.py` — check the condition and `set(...) <= allowed_ids`
every question says "I don't know"	`RETRIEVAL_MIN_SCORE` in `.env` is too high; keep 2.0 for `bm25`
`ModuleNotFoundError: assistant_core`	activate `.venv` and run `pip install -e .` in `lab06/starter/`
`compare_paths.py` cannot find `eval/cases.jsonl`	run it from the `starter/` folder
Ollama: `ConnectError`	start Ollama, and check `ollama list` shows the model
Ollama answers slowly	normal for the first call; laptops without a GPU are slower

Glossary

Word	Meaning
AI / machine learning	programs that learn patterns from examples instead of following only hand-written rules
LLM	a large language model: predicts text word by word
prompt	the instructions and data we send to the model
token	a piece of a word; the unit models read, write and charge for
hallucination	a fluent but false answer
inverted index	a dict from each word to the documents containing it
BM25	a standard formula for scoring how well a document matches a query
chunk	a small piece of a document (here: one section) that we search and cite
RAG	search our documents first, then let the model answer from what was found
structured output	forcing the model to answer in a fixed data shape (JSON)
grounding	the answer is supported by the retrieved sources
abstain / escalate	say "I don't know" / send the student to a human
stub	a fake model used for learning and testing

Self-Check

1. In one sentence, what does an LLM do?
2. What is stored in an inverted index?
3. Why do we check that the model cites only retrieved chunks?
4. Why do we abstain *before* calling the model when search finds nothing?
5. Why are fees computed by code and not by the model?

Answers. 1) It predicts the next piece of text, again and again, to produce an answer. 2) For each word, the set of documents (or chunks) that contain it. 3) A citation of something we did not give it means the answer was probably invented. 4) The model would guess from memory; refusing is safer, faster and free. 5) Money must be exact and the same every time; a model can make mistakes.

Resources

- Hugging Face — LLM Course, chapter 1: *Transformer models* (huggingface.co/learn/llm-course) — optional, for curiosity
- Ollama — getting started (ollama.com, docs.ollama.com)
- Course materials — theory Week 3 (retrieval) and Week 4 (AI as a component)